

12주차 예습과제

📖 자연어처리와 컴퓨터비전 심층학습 7장 (~GPT)

7. 트랜스포머

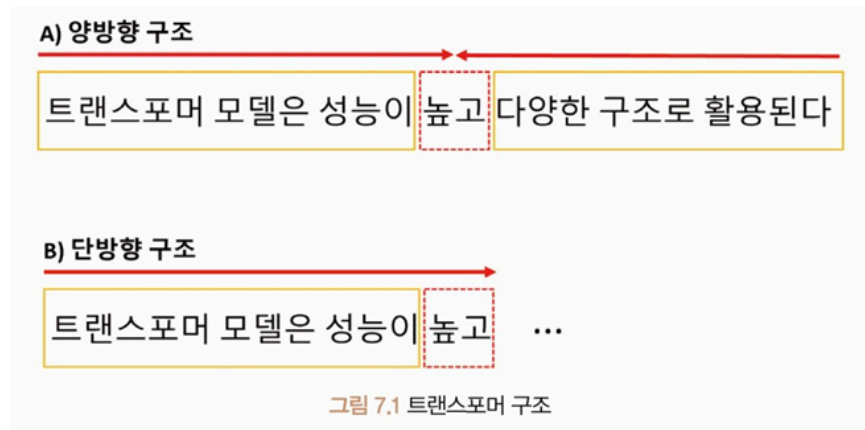
- **트랜스포머(Transformer)**

- 2017년 소개된 신경망 아키텍처
- 병렬로 입력 시퀀스를 처리, 긴 시퀀스의 경우 순환 신경망 모델보다 더 빠르고 효율적으로 처리

⇒ **순차 처리(Sequential Processing)**나 **반복 연결(Recurrent Connections)**에 의존하지 않고 입력 토큰 간의 관계를 직접 처리하고 이해할 수 있도록 하는 **셀프 어텐션(Self-Attention)**을 기반으로 함

→ 모델이 재귀나 합성곱 연산 없이 입력 토큰 간의 관계 직접 모델링 가능

- 대용량 데이터셋에서 효율적으로 학습 가능 → 데이터의 양 많은 기계 번역 같은 작업에 적합
- 언어 모델링, 텍스트 분류, 광범위한 자연어 처리 작업, 기계번역, 언어 모델링, 텍스트 요약 작업
- 트랜스포머 기반 모델들은 **오토 인코딩(Auto-Encoding)**, **자기 회귀(Auto Regressive)** 방식으로 학습
- **오토 인코딩**
 - 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업 수행
 - 예측되는 토큰의 양 옆에 있는 토큰들을 참조하기 때문에 양방향 구조 → **인코더(Encoder)**
- 트랜스포머 구조



- A) '트랜스포머 모델은 성능이', '다양한 구조로 활용된다': 입력, '높고': 출력 양방향 구조

→ 반면 자기 회귀 방식은 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞추는 작업을 수행

- B) '트랜스포머 모델은 성능이': 입력, '높고': 출력 단방향 구조

→ 예측되는 토큰의 왼쪽에 있는 토큰들만 참조하기 때문에 단방향 구조 → **디코더 (Decoder)**

⇒ 소개하는 트랜스포머 모델들은 양방향성 구조의 인코더(오토 인코딩) 또는 단방향성 디코더(자기 회귀)를 사용하는 공통적 특징

표 7.1 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

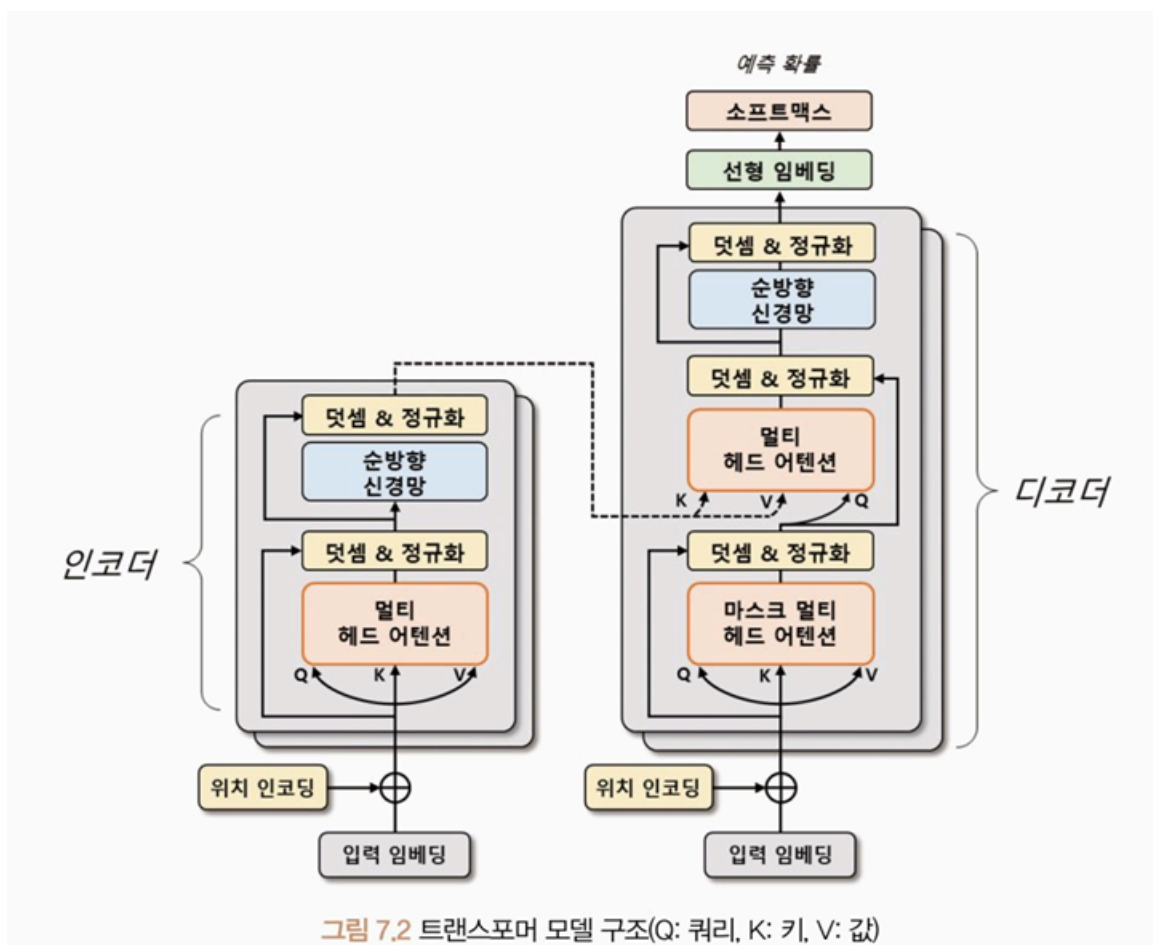
Transformer

• 트랜스포머(Transformer)

- 기계 번역, 챗봇, 음성 인식 등 다양한 자연어 처리 분야에서 성과 내는 딥러닝 모델
- **어텐션 메커니즘(Attention Mechanism)**만을 사용하여 시퀀스 임베딩을 표현

• 트랜스포머의 어텐션 메커니즘

- 인코더와 디코더 간의 상호작용 → 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥을 이해하고 적절한 출력을 생성
- 인코더 - 입력 시퀀스를 임베딩해 고차원 벡터로 변환
- 디코더 - 인코더의 출력을 입력으로 받아 출력 시퀀스를 생성
- 어텐션 메커니즘은 인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중
 - 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지를 파악하여 번역이나 요약문 생성과 관련된 작업 등을 수행
- 트랜스포머 모델은 기존의 순환 신경망 기반 모델보다 학습 속도가 빠르고, 병렬 처리가 가능해 대규모 데이터셋에서 높은 성능 보임
- 임베딩 과정에서 문장의 전체 정보를 고려 → 문장의 길이 길어지더라도 성능이 유지되는 장점
- 트랜스포머 모델 구조



- 인코더와 디코더는 두 부분으로 구성되어 있으며, 각각 N개의 **트랜스포머 블록 (Transformer Block)**으로 구성
 - 이 블록은 **멀티 헤드 어텐션(Multi-Head Attention)**과 순방향 신경망으로 이루어짐
- 멀티 헤드 어텐션
 - 입력 시퀀스에서 **쿼리(Query), 키(Key), 값(Value)** 벡터를 정의해 입력 시퀀스들의 관계를 **셀프 어텐션(Self-Attention)**하는 벡터 표현 방법
 - 쿼리와 각 키의 유사도를 계산, 해당 유사도를 가중치로 사용하여 값 벡터를 합산
 - 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체

⇒ 입력 시퀀스의 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신함
- 순방향 신경망
 - 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용
 - 여러 개의 선형 계층으로 구성, 앞선 순방향 신경망의 구조와 동일하게 입력 벡터에 가중치를 곱하고 편향을 더하여 활성화 함수를 적용
 - 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신
- 트랜스포머에서는 입력 시퀀스 데이터를 **소스(Source)**와 **타겟(Target)** 데이터로 나눠 처리

Ex) 영어를 한글로 번역하는 경우 - 생성하는 언어인 한국어를 타겟, 참조하는 언어인 영어를 소스 데이터로 정의

 - 인코더는 소스 시퀀스 데이터를 **위치 인코딩(Positional Encoding)**된 입력 임베딩으로 트랜스포머 블록의 출력 벡터를 생성
 - 출력 벡터는 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성됨
 - 디코더는 **마스크 멀티 헤드 어텐션(Masked Multi-Head Attention)**을 사용해 타겟 시퀀스 데이터를 순차적으로 생성
 - 이때 디코더 입력 시퀀스들의 관계를 고도화하기 위해 인코더의 출력 벡터 정보를 참조
 - 최종적으로 생성된 디코더 출력 벡터는 선형 임베딩으로 재표현되어 이미지나 자연어 모델에 활용됨

입력 임베딩과 위치 인코딩

- 트랜스포머 모델은 입력 시퀀스를 병렬 구조로 처리하기 때문에 단어의 순서 정보를 제공 X
 - 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영해야 함
 - ⇒ 인코딩 방식 사용
- 위치 인코딩
 - 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
 - 각 단어의 위치 정보를 나타내는 벡터를 더하여 임베딩 벡터에 위치 정보를 반영
 - 위치 인코딩 벡터를 추가함으로써 모델은 단어의 순서 정보를 학습할 수 있게 됨
- $\sin$ 함수, $\cos$ 함수를 사용해 임베딩 벡터와 위치 정보가 결합된 최종 입력 벡터를 생성
 - 각 토큰의 위치를 각도로 표현해 $\sin$ 함수, $\cos$ 함수로 위치 인코딩 벡터를 계산
 - 토큰의 위치마다 동일한 임베딩 벡터를 사용 X, 각 토큰이 위치 정보를 모델이 학습 가능



```
#위치 인코딩
import math
import torch
from torch import nn
from matplotlib import pyplot as plt

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

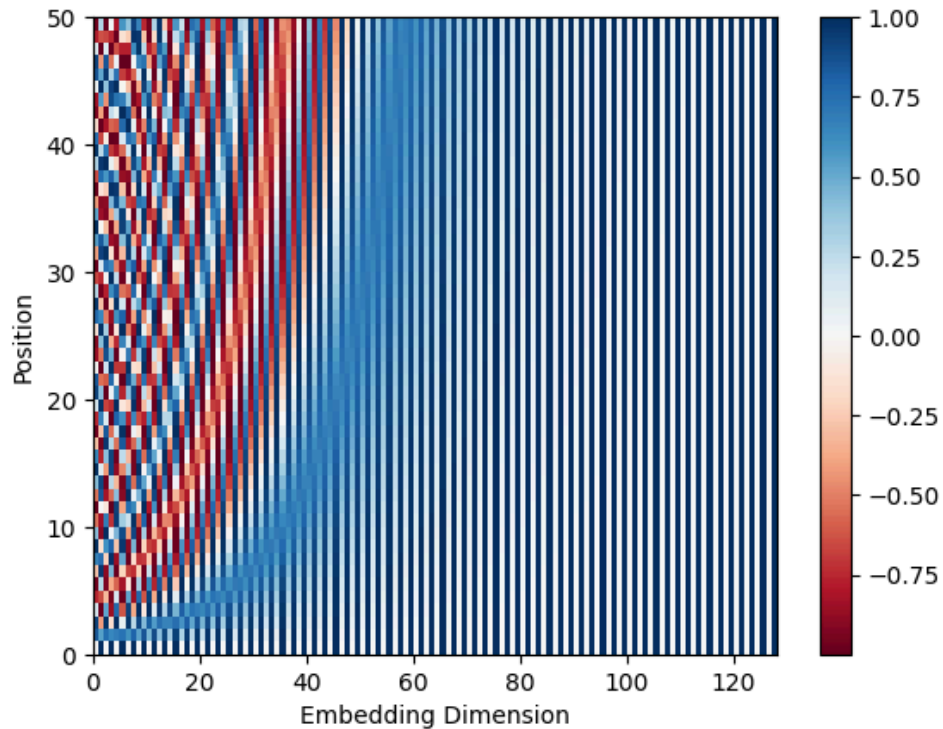
        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )

        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[: x.size(0)]
        return self.dropout(x)

encoding = PositionalEncoding(d_model=128, max_len=50)

plt.pcolormesh(encoding.pe.numpy().squeeze(), cmap="RdBu")
plt.xlabel("Embedding Dimension")
plt.xlim((0,128))
plt.ylabel("Position")
plt.colorbar()
plt.show()
```



- `PositionalEncoding` 클래스
 - 입력 임베딩 차원(`d_model`), 최대 시퀀스(`max_len`)을 입력받음
 - 입력 시퀀스의 위치마다 $\sin, \cos$ 함수로 위치 인코딩 계산
 - `pe` 변수의 텐서 차원은 `[50, 1, 128]` → [최대 시퀀스, 1, 입력 임베딩의 차원] 의미
 - 출력 결과에서 위치별 임베딩 차원이 주기적인 값으로 구성되는 것 확인 가능
 - 산출된 위치 인코딩은 입력 임베딩과 더해진 후 드롭아웃 적용
 - `register_buffer` 메서드는 모델이 매개변수 갱신하지 않도록 설정
- 위치 인코딩 계산 방식

수식 7.1 위치 인코딩

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

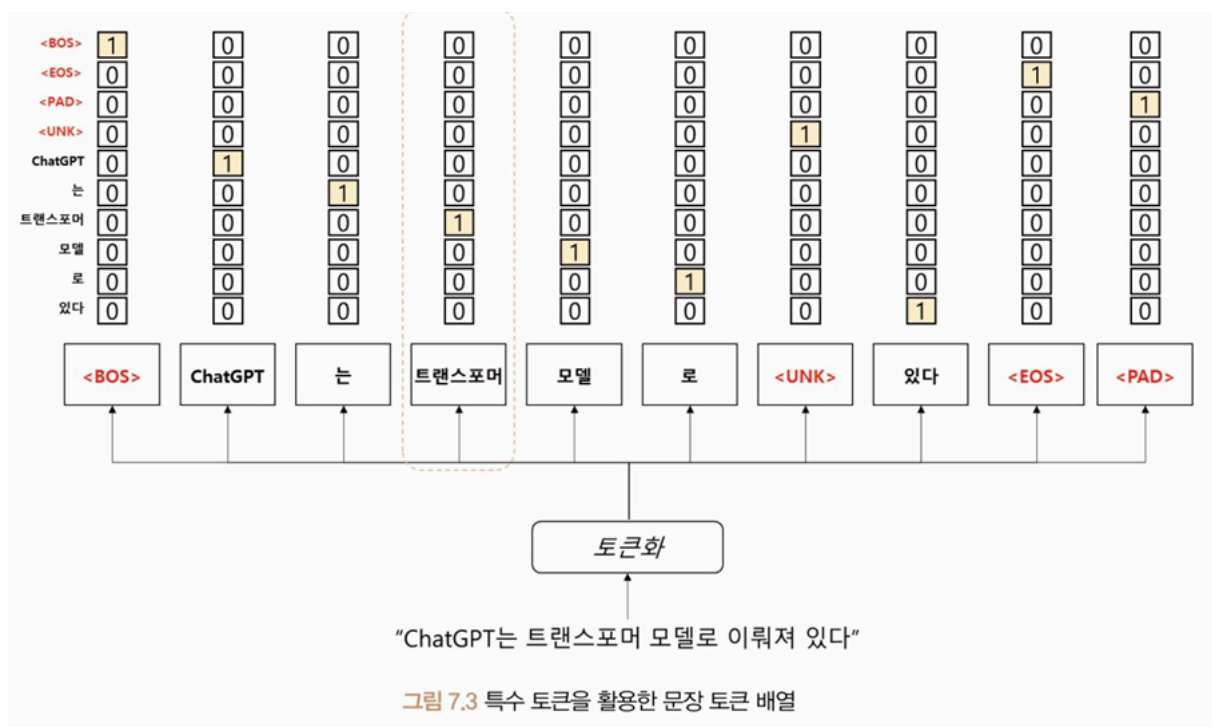
- pos 는 입력 시퀀스에서 해당 단어의 위치, i 는 임베딩 벡터의 차원 인덱스
- 차원의 인덱스가 짝수면 $\sin$, 홀수면 $\cos$ 함수 수식 적용

- 각도 정보로 변환하기 위한 스케일링 인자 → 각 위치에 대한 주기적인 신호 생성

특수 토큰

- 트랜스포머는 단어 토큰 이외의 특수 토큰을 활용하여 문장 표현
- 특수 토큰은 입력 시퀀스의 시작과 끝 나타내거나 **마스킹(Masking)** 영역으로 사용
- 모델이 입력 시퀀스의 시작과 끝 인식할 수 있게 하며, 마스킹을 통해 일부 입력 무시 가능

Ex) 번역 모델 - 디코더의 입력 시퀀스에서 현재 위치 이후의 토큰을 마스크해 이전 토큰만을 참조

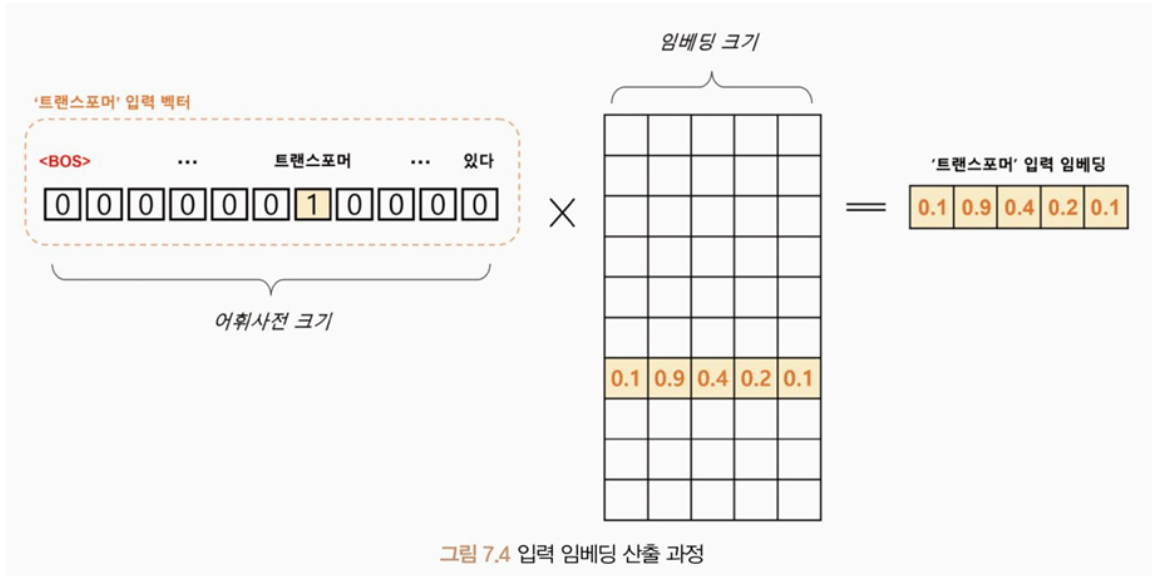


- **BOS(Beginnig of Sentence), EOS(End of Sentence), UNK(Unknown), PAD(Padding)** 토큰

→ 모두 특수 토큰으로 자연어 처리에서 일반적으로 사용됨

- BOS 토큰: 문장의 시작 나타냄
- EOS 토큰: 문장의 종료 나타냄
 - 문장의 시작과 끝 명확히 하기 위해 사용
- UNK 토큰: 어휘 사전에 없는 단어로, 모르는 단어를 의미
 - 모델이 이전에 본 적 없는 단어 처리할 때 사용

- PAD 토큰: 모든 문장을 일정한 길이로 맞추기 위해 사용
→ 짧은 문장의 빈 공간을 채울 때 사용
- 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현



- 입력 임베딩으로 변환되는 과정은 Word2Vec과 동일
 - 어휘 사전의 크기를 V , 입력 임베딩 차원을 d 라 했을 때 '트랜스포머'의 원-핫 벡터는 $[1, V]$ 크기
 - 원-핫 벡터는 임베딩 행렬 $[V, d]$ 에 의해 $[1, d]$ 크기의 벡터로 변환됨
- ⇒ N 개의 문장이 최대 S 개의 토큰 길이를 가질 때 $[N, S, V]$ 크기의 원-핫 벡터 텐서는 $[N, S, d]$ 크기의 임베딩 텐서로 변환됨
- 임베딩 텐서는 입력으로 사용되며, 트랜스포머의 모든 계층에서 공유하는 텐서

트랜스포머 인코더

- 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행
- 각 인코더 계층은 멀티 헤드 어텐션과 순방향 신경망으로 구성, 입력 데이터에 대한 정보를 추출하고 다음 계층으로 전달
 - 인코더 계층에서 위치 정보를 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더해줌
 - 산출된 인코더 계층의 출력은 디코더 계층으로 전달됨

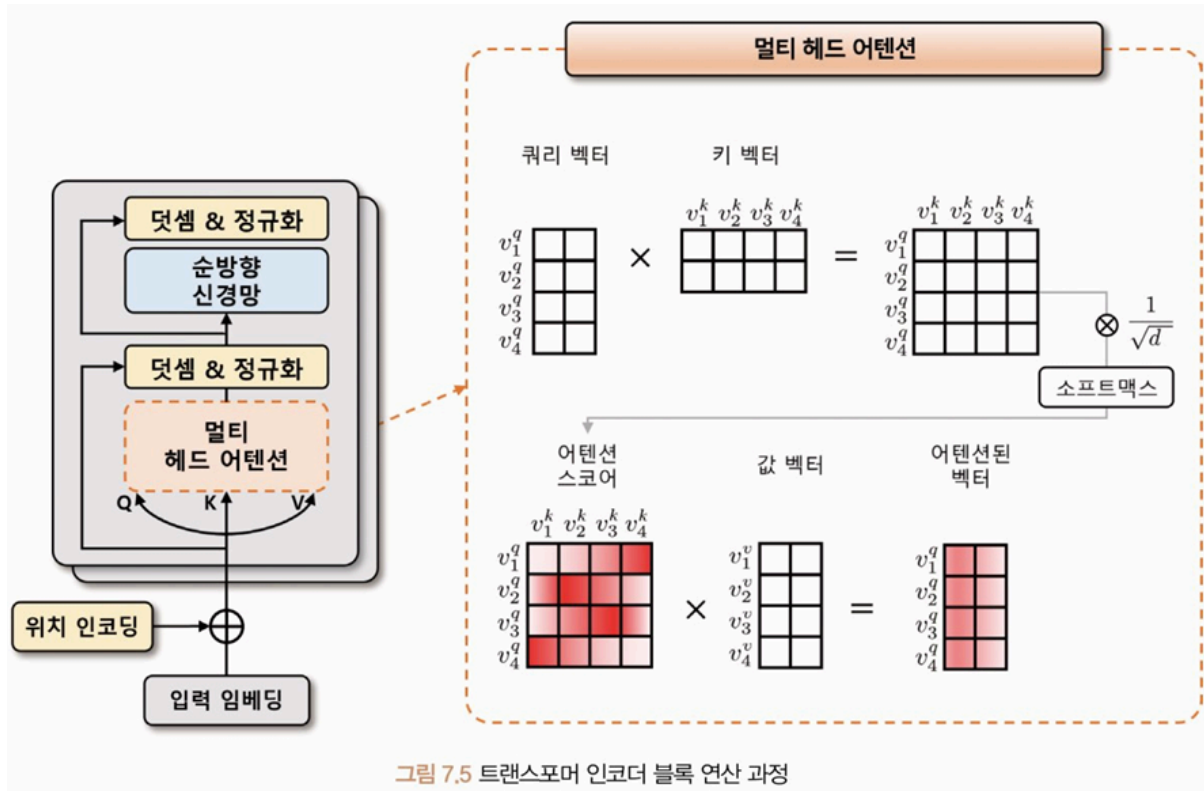


그림 7.5 트랜스포머 인코더 블록 연산 과정

- 트랜스포머 인코더는 위치 인코딩이 적용된 소스 데이터의 입력 임베딩을 입력받음
 - 멀티 헤드 어텐션 단계에서 입력 텐서 차원이 $[N, S, d]$ 라면 입력 임베딩은 선형 변환을 통해 3개의 임베딩 벡터를 생성
 - 생성된 3개의 벡터를 각각 쿼리(Q), 키(K), 값(V) 벡터라고 정의
 - 쿼리 벡터(v^q)
 - 현재 시점에서 참조하고자 하는 정보의 위치 나타내는 벡터
 - 인코더의 각 시점마다 생성
 - 현재 시점에서 질문이 되는 벡터를 의미하며, 이 벡터를 기준으로 다른 시점의 정보를 참조
 - 키 벡터(v^k)
 - 쿼리 벡터와 비교되는 대상으로, 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터
 - 인코더의 각 시점에서 생성
 - 값 벡터(v^v)
 - 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할

- 쿼리, 키, 값 벡터는 초기에 비슷한 값 가지지만 모델 학습을 통해 의도한 의미의 값을 갖게 됨
- 쿼리와 키 벡터의 연관성은 내적 연산으로 $[N, S, S]$ 어텐션 스코어 맵을 사용
- 쿼리, 키 벡터를 사용한 어텐션 스코어 계산식

수식 7.2 어텐션 스코어 계산식

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

- 쿼리, 키, 값 벡터를 내적해 어텐션 스코어를 구하고 이 스코어값에 $\sqrt{d}$ (벡터 차원의 제곱근) 만큼 나눠 보정
 - 보정값은 벡터 차원이 커질 때 스코어값이 같이 커지는 문제를 완화
- 보정된 어텐션 스코어를 소프트맥스 함수를 이용해 확률적으로 재표현하고, 이를 값 벡터와 내적하여 셀프 어텐션 된 벡터를 생성
 - 이 과정 반복해 셀프 어텐션된 스코어 맵 생성
- **멀티 헤드(Multi-Head)**는 이러한 셀프 어텐션을 여러 번 수행해 여러 개의 헤드 만듦
 - 각각의 헤드가 독립적으로 어텐션을 수행하고 그 결과를 합침
 - 입력받는 $[N, S, d]$ 텐서에 k 개의 셀프 어텐션 벡터를 생성하고자 한다면 헤드에 대한 차원 축을 생성해 $[N, k, S, d/k]$ 텐서 형태를 구성 - k 개의 셀프 어텐션된 $[N, S, d/k]$ 텐서를 의미
 - 생성된 k 개의 셀프 어텐션 벡터는 임베딩 차원 축으로 다시 **병합(concatenation)**되어 $[N, S, d]$ 의 형태로 출력
- **덧셈 & 정규화**
 - 멀티 헤드 어텐션을 통과하기 이전의 입력값 $[N, S, d]$ 텐서와 통과한 이후의 출력값 $[N, S, d]$ 텐서를 더함 → 학습 시 발생하는 기울기 소실 완화
 - 이 값에 임베딩 차원 축으로 정규화하는 계층 정규화를 적용
- **순방향 신경망**
 - 선형 임베딩과 ReLU로 이뤄진 인공신경망 또는 1차원 합성곱이 사용됨
 - 이어서 다시 덧셈 & 정규화 과정 수행
- 트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성

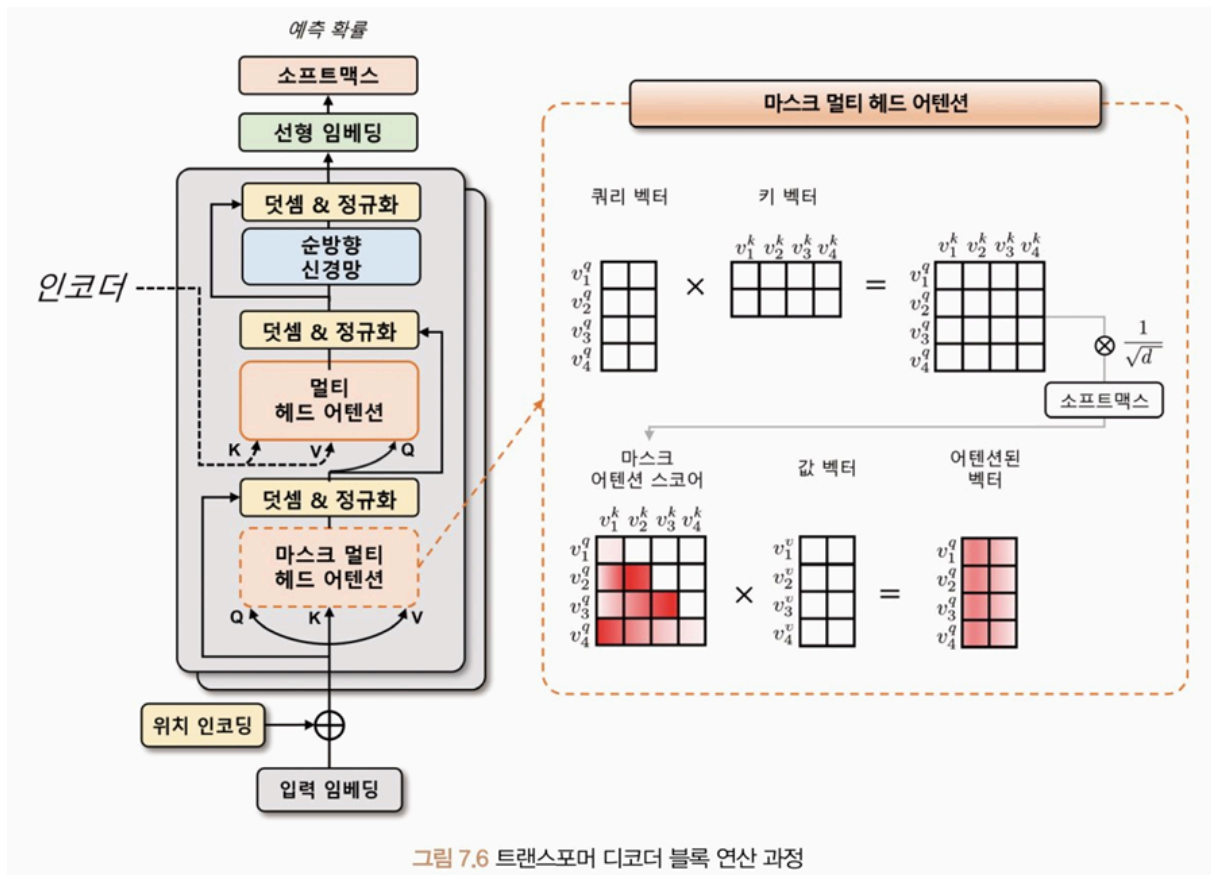
- 이전 블록에서 출력된 벡터는 다음 블록의 입력으로 전달되어 인코더 블록을 통과하면서 입력 시퀀스의 정보가 추상화됨
- 마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용 → 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터로 활용

⇒ 이런 방식으로 인코더와 디코더는 서로 정보를 공유

트랜스포머 디코더

- 위치 인코딩이 적용된 타겟 데이터의 입력 임베딩을 입력받음
- 디코더의 입력 임베딩에 위치 정보를 추가함으로써 디코더가 입력 시퀀스의 순서 정보를 학습 가능
- 인코더의 멀티 헤드 어텐션 모듈은 **인과성(Casuality)**를 반영한 마스크 멀티 헤드 어텐션 모듈로 대체됨
 - 어텐션 스코어 맵을 계산할 때 첫 번째 쿼리 벡터는 첫 번째 키 벡터만, 두 번째 쿼리 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크를 씌움
 - 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 되며, 인과성이 보장됨
 - 마스크 영역에 수치적으로 굉장히 작은 값인 $-\inf$ 마스크를 더해줌으로써 해당 영역의 어텐션 스코어값이 0에 가까워짐
 - 소프트맥스 계산 시 해당 영역의 어텐션 가중치가 0이 되므로, 마스크 영역 이전에 있는 입력 토큰들에 대한 정보를 참조하지 않게 됨

⇒ 마스크 멀티 헤드 어텐션 모듈은 인코더의 멀티 헤드 어텐션 모듈과 유사하지만, 인과성을 보장하면서 셀프 어텐션을 수행할 수 있게 됨



- 디코더의 멀티 헤드 어텐션에서는 타겟 데이터가 쿼리 벡터로 사용되며, 인코더의 소스 데이터가 키와 값 벡터로 사용됨
 - 쿼리 벡터는 타겟 데이터의 위치 정보를 포함한 입력 임베딩과 위치 인코딩을 더한 벡터가 됨
 - 이후 쿼리, 키, 값 벡터를 이용해 어텐션 스코어 맵을 계산하고, 소프트맥스 함수를 적용하여 어텐션 가중치를 구함
 - 최종으로 어텐션 가중치와 값 벡터를 가중합하여 멀티 헤드 어텐션의 출력 벡터를 얻을 수 있음
- 디코더에서는 셀프 어텐션을 방지하기 위해 마스킹을 적용
- 디코더는 여러 개의 트랜스포머 디코더 블록으로 구성
 - 이전 트랜스포머 디코더 블록의 산출물은 다음 트랜스포머 디코더 블록의 입력으로 전달됨
 - 이전 시점에서의 정보를 현재 시점에서 활용할 수 있게 됨
 - 마지막 디코더 블록의 출력 텐서 $[N, S, d]$ 에 선형 변환 및 소프트맥스 함수를 적용해 각 타겟 시퀀스 위치마다 예측 확률을 계산할 수 있게 됨
- 디코더는 타겟 데이터를 추론할 때 토큰 또는 단어를 순차적으로 생성시키는 모델

- 입력 토큰을 순차적으로 나타내는 방법은 빈 값을 의미하는 PAD 토큰을 사용하는 것
- 'ChatGPT는 트랜스포머 모델로 이뤄져 있다'라는 문장을 추론할 때 디코더의 입력과 출력 예시

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD], [PAD], [PAD], [PAD]

모델 실습

- 파이토치에서 제공하는 트랜스포머 모델을 활용해 영어-독일어 번역 모델 구성
- **Multi30k** 데이터세트 사용
 - 자연어 처리를 위한 대규모 다국어 데이터세트
 - 영어-독일어 **병렬 말뭉치(Parallel corpus)**로 약 30,000개의 데이터를 제공
 - **토치 데이터(torchdata)**와 **토치 텍스트(torchtext)** 라이브러리로 다운 가능



#토치 데이터 및 토치 텍스트 라이브러리 설치
!pip install torchdata torchtext portalocker

- 토치 데이터 라이브러리
 - 대규모 데이터셋을 다루기 쉽게 데이터를 불러오고 변환 및 배치하는 API 제공
- 토치 텍스트 라이브러리
 - 파이토치를 위한 텍스트 처리 라이브러리
 - 다양한 언어 모델링 작업에 대해 사전 처리 및 데이터셋 관리를 단순화하기 위한 다양한 도구와 기능 제공
- 포르타락커(portalocker) 라이브러리
 - 파이썬에서 파일 락을 관리하기 위한 라이브러리
 - 파일 락을 사용해 여러 프로세스 간에 동시에 파일을 수정하거나 읽는 것을 방지
 - Multi30k 데이터셋 다운 및 압축 해제 과정에서 내부적으로 사용



```
#데이터세트 다운로드 및 전처리
from torchtext.datasets import Multi30k
from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator

def generate_tokens(text_iter, language):
    language_index = {SRC_LANGUAGE: 0, TGT_LANGUAGE: 1}

    for text in text_iter:
        yield token_transform[language](text[language_index[language]])

SRC_LANGUAGE = "de"
TGT_LANGUAGE = "en"
UNK_IDX, PAD_IDX, BOS_IDX, EOS_IDX = 0, 1, 2, 3
special_symbols = ["<unk>", "<pad>", "<bos>", "<eos>"]

token_transform = {
    SRC_LANGUAGE: get_tokenizer("spacy", language="de_core_news_sm"),
    TGT_LANGUAGE: get_tokenizer("spacy", language="en_core_web_sm"),
}
print("Token Transform:")
print(token_transform)

vocab_transform = {}
for language in [SRC_LANGUAGE, TGT_LANGUAGE]:
    train_iter = Multi30k(split="train", language_pair=(SRC_LANGUAGE, TGT_LANGUAGE))
    vocab_transform[language] = build_vocab_from_iterator(
        generate_tokens(train_iter, language),
        min_freq=1,
        specials=special_symbols,
        special_first=True,
    )
for language in [SRC_LANGUAGE, TGT_LANGUAGE]:
    vocab_transform[language].set_default_index(UNK_IDX)

print("Vocab Transform:")
print(vocab_transform)
```

```
Token Transform:
{'de': functools.partial(<function _spacy_tokenize at 0x79f7e3a5c400>, spacy=<spacy.lang.de.German object at 0x79f7d697cfe0>),
 'en': functools.partial(<function _spacy_tokenize at 0x79f7e3a5c400>, spacy=<spacy.lang.en.English object at 0x79f7d697cfe0>)}
Vocab Transform:
{'de': Vocab(), 'en': Vocab()}
```

- 독일어 말뭉치(`de_core_news_sm`)와 영어 말뭉치(`en_core_web_sm`)에 대해 각각 토큰나이저와 어휘 사전 생성
- `get_tokenizer` 함수
 - 사용자가 지정한 토큰나이저를 가져오는 유틸리티 함수
 - spaCy 라이브러리로 사전 학습된 모델 가져와 `token_transform` 변수에 저장

- `vocab_transform` 변수
 - 토큰을 인덱스로 변환시키는 함수 저장
 - `Multi30k` 데이터세트 활용해 (독일어, 영어)의 튜플 형식으로 데이터 불러옴
- 데이터를 불러왔다면 `build_vocab_from_iterator` 함수와 `generate_tokens` 함수로 언어별 어휘 사전 생성
 - `build_vocab_from_iterator` 함수
 - 생성된 토큰을 이용해 단어 집합 생성
 - 최소 빈도(`min_freq`): 토큰화된 단어들의 최소 빈도수 지정
 - 특수 토큰(`specials`): 트랜스포머에 활용하는 특수 토큰을 지정, `special_first` 매개변수가 참인 경우 특수 토큰을 단어 집합의 맨 앞에 추가
- `set_default_index` 메서드
 - 인덱스의 기본값을 설정 → 어휘 사전에 없는 토큰인 `<unk>`의 인덱스를 할당



#트랜스포머 모델 구성

```
import math
import torch
from torch import nn

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )

        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[: x.size(0)]
        return self.dropout(x)
```

```

class TokenEmbedding(nn.Module):
    def __init__(self, vocab_size, emb_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_size)
        self.emb_size = emb_size

    def forward(self, tokens):
        return self.embedding(tokens.long()) * math.sqrt(self.emb_size)

class Seq2SeqTransformer(nn.Module):
    def __init__(
        self,
        num_encoder_layers,
        num_decoder_layers,
        emb_size,
        max_len,
        nhead,
        src_vocab_size,
        tgt_vocab_size,
        dim_feedforward,
        dropout=0.1,
    ):
        super().__init__()
        self.src_tok_emb = TokenEmbedding(src_vocab_size, emb_size)
        self.tgt_tok_emb = TokenEmbedding(tgt_vocab_size, emb_size)
        self.positional_encoding = PositionalEncoding(
            d_model=emb_size, max_len=max_len, dropout=dropout
        )
        self.transformer = nn.Transformer(
            d_model=emb_size,
            nhead=nhead,
            num_encoder_layers=num_encoder_layers,
            num_decoder_layers=num_decoder_layers,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
        )
        self.generator = nn.Linear(emb_size, tgt_vocab_size)

```

```

def forward(
    self,
    src,
    trg,
    src_mask,
    tgt_mask,
    src_padding_mask,
    tgt_padding_mask,
    memory_key_padding_mask,
):
    src_emb = self.positional_encoding(self.src_tok_emb(src))
    tgt_emb = self.positional_encoding(self.tgt_tok_emb(trg))
    outs = self.transformer(
        src=src_emb,
        tgt=tgt_emb,
        src_mask=src_mask,
        tgt_mask=tgt_mask,
        memory_mask=None,
        src_key_padding_mask=src_padding_mask,
        tgt_key_padding_mask=tgt_padding_mask,
        memory_key_padding_mask=memory_key_padding_mask
    )
    return self.generator(outs)

def encode(self, src, src_mask):
    return self.transformer.encoder(
        self.positional_encoding(self.src_tok_emb(src)), src_mask
    )

def decode(self, tgt, memory, tgt_mask):
    return self.transformer.decoder(
        self.positional_encoding(self.tgt_tok_emb(tgt)), memory, tgt_mask
    )

```

- **Seq2SeqTransformer** 클래스
 - **TokenEmbedding** 클래스로 소스 데이터와 입력 데이터를 입력 임베딩으로 변환하여 **src_tok_emb** 와 **tgt_tok_emb** 를 생성
 ⇒ 소스와 타깃 데이터의 어휘 사전 크기를 입력받아 트랜스포머 임베딩 크기로 변환, 이 입력 임베딩에 예제에서 정의한 **PositionalEncoding** 을 적용해 트랜스포머 블록에 입력
- 트랜스포머 블록(**self.transformer**)
 - 파이토치에서 제공하는 트랜스포머(**Transformer**) 클래스를 적용
 - 트랜스포머의 인코더와 디코더는 **encoder_layers** 변수의 값으로 구성
- 순방향 메서드 마지막에 적용되는 **generator** 는 마지막 트랜스포머 디코더 블록에서 산출되는 벡터를 선형 변환해 어휘 사전에 대한 로짓(**logit**)을 생성



```
#트랜스포머 클래스
transformer = torch.nn.Transformer(
    d_model=512,
    nhead=8,
    num_encoder_layers=6,
    num_decoder_layers=6,
    dim_feedforward=2048,
    dropout=0.1,
    activation=torch.nn.functional.relu,
    layer_norm_eps=1e-05,
)
```

- 임베딩 차원(**d_model**)
 - 트랜스포머 모델의 입력과 출력 차원의 크기 정의
 - 임베딩 차원 크기와 동일
- 헤드(**nhead**)
 - 멀티 헤드 어텐션의 헤드 개수 정의
 - 헤드의 개수는 모델이 어텐션을 수행하는 방법을 결정, 헤드의 개수가 많을수록 모델의 병렬 처리 능력이 증가/모델 매개변수의 수도 증가
 - 인코더 계층 개수(**num_encoder_layers**), 디코더 계층 개수(**num_decoder_layers**)
 - 인코더와 디코더의 계층 수 의미
 - 모델의 복잡도와 성능에 영향
 - 계층 개수가 많을수록 더 복잡한 문제를 해결 가능, 너무 많은 경우에는 과대적합 위험
 - 순방향 신경망 크기(**dim_feedforward**)
 - 순방향 신경망의 은닉층 크기 정의
 - 순방향 신경망 계층은 트랜스포머 계층의 각 입력 위치에 독립적으로 적용됨
 - 모델의 복잡도와 성능에 영향
 - 드롭아웃(**dropout**)
 - 인코더와 디코더 계층에 적용되는 드롭아웃 비율 적용

- 활성화 함수(**activation**)
 - 순방향 신경망에 적용되는 활성화 함수 의미
 - 파이토치 함수 형태로 입력 가능
- 계층 정규화 입실론(**layer_norm_eps**)
 - 계층 정규화를 수행할 때 분모에 더해지는 입실론 값을 정의



#트랜스포머 순방향 메서드

```
output = transformer.forward(  
    src,  
    tgt,  
    src_mask=None,  
    tgt_mask=None,  
    memory_mask=None,  
    src_key_padding_mask=None,  
    tgt_key_padding_mask=None,  
    memory_key_padding_mask=None,  
)
```

- 소스(**src**), 타겟(**tgt**)
 - 인코더와 디코더에 대한 시퀀스
 - [소스(타겟) 시퀀스 길이, 배치 크기, 임베딩 차원] 형태의 데이터 입력
- 소스 마스크(**src_mask**), 타겟 마스크(**tgt_mask**)
 - 소스와 타겟 시퀀스의 마스크
 - [소스(타겟) 시퀀스 길이, 시퀀스 길이] 형태의 데이터 입력
 - 마스크의 값이 0이라면 해당 위치에서는 모든 입력 단어가 동일한 가중치 갖고 어텐션이 수행, 1이라면 모든 입력 단어의 가중치가 0으로 설정돼 어텐션 연산 수행 X
 - -inf라면 해당 위치에서는 어텐션 연산 결과에 0으로 가중치가 부여돼 마스크된 위치의 정보를 모델이 무시하게 만듦
 - +inf라면 모든 입력 단어에 무한대의 가중치가 부여돼 어텐션 연산 결과가 해당 위치에 대한 정보만으로 구성됨
 - 일반적으로 +inf는 적용 X, 어떤 특정 단어나 위치에 대해 모델이 특별한 관심 가지도록 할 때만 사용
- 메모리 마스크(**memory_mask**)
 - 인코더 출력의 마스크
 - [타겟 시퀀스 길이, 소스 시퀀스 길이]의 형태
 - 메모리 마스크의 값이 0인 위치에서는 어텐션 연산이 수행 X
- 소스, 타겟, 메모리 키 패딩 마스크(**key_padding_mask**)

- 소스, 타깃, 메모리 시퀀스에 대한 패딩 마스크 의미
- [배치 크기, 소스(타깃) 시퀀스 길이] 형태의 데이터 입력
- 메모리 키 패딩 마스크는 소스 키 패딩 마스크와 동일한 형태의 데이터 입력
- 키 패딩 마스크
 - 입력 시퀀스에서 패딩 토큰이 위치한 부분 가리키는 이진 마스크
 - 패딩 토큰이 실제 의미를 가지지 않는 것으로 간주되어 해당 위치의 어텐션 연산 결과에 대한 가중치 0으로 만들
- 순방향 메서드는 인스턴스의 설정과 입력 시퀀스를 통해 타깃 시퀀스의 임베딩 텐서를 반환
 - [타깃 시퀀스 길이, 배치 크기, 임베딩 차원] 반환
- 현재 클래스에서는 어휘 사전에 대한 로깅을 생성 → 임베딩 차원이 타깃 데이터의 어휘 사전 크기로 변경됨



#트랜스포머 모델 구조

```
from torch import optim

BATCH_SIZE = 128
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

model = Seq2SeqTransformer(
    num_encoder_layers=3,
    num_decoder_layers=3,
    emb_size=512,
    max_len=512,
    nhead=8,
    src_vocab_size=len(vocab_transform[SRC_LANGUAGE]),
    tgt_vocab_size=len(vocab_transform[TGT_LANGUAGE]),
    dim_feedforward=512,
).to(DEVICE)
criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX).to(DEVICE)
optimizer = optim.Adam(model.parameters())

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("| | L", sssub_name)

src_tok_emb
L embedding
tgt_tok_emb
L embedding
positional_encoding
L dropout
transformer
L encoder
| L layers
| | L 0
| | L 1
| | L 2
| L norm
L decoder
| L layers
| | L 0
| | L 1
| | L 2
| L norm
generator
```

- Seq2SeqTransformer 클래스
 - 입력 임베딩(src_tok_emb , tgt_tok_emb), 위치 인코딩(positional_encoding), 트랜스포머 블록(transformer), 로짓 생성(generator)으로 구성

- 출력 결과에서 확인할 수 있듯 인코더와 디코더가 각각 세 개(0, 1, 2)의 계층으로 구성
 - 앞선 그림 7.2의 구조에서 인코더와 디코더가 세 번 반복되고, 로짓 생성이 추가된 구조
- 손실 함수는 교차 엔트로피 함수 적용
 - 무시되는 색인(`ignore_index`) 값을 패딩 토큰(`PAD_IDX`)을 할당해 모델이 학습하는 동안 무시해야 할 클래스 레이블 지정
 - 패딩 토큰은 모델 학습에 사용되지 않으므로 해당 토큰에 대한 레이블을 무시하고 모델이 해당 클래스를 학습하지 않게 함
 - 이 클래스에 대한 손실은 계산 X, 모델이 해당 클래스 예측하더라도 올바른 예측으로 간주 X



```
#배치 데이터 생성
from torch.utils.data import DataLoader
from torch.nn.utils.rnn import pad_sequence

def sequential_transforms(*transforms):
    def func(txt_input):
        for transform in transforms:
            txt_input = transform(txt_input)
        return txt_input
    return func

def input_transform(token_ids):
    return torch.cat(
        (torch.tensor([BOS_IDX]), torch.tensor(token_ids), torch.tensor([EOS_IDX]))
    )

def collator(batch):
    src_batch, tgt_batch = [], []
    for src_sample, tgt_sample in batch:
        src_batch.append(text_transform[SRC_LANGUAGE](src_sample.rstrip("\n")))
        tgt_batch.append(text_transform[TGT_LANGUAGE](tgt_sample.rstrip("\n")))

    src_batch = pad_sequence(src_batch, padding_value=PAD_IDX)
    tgt_batch = pad_sequence(tgt_batch, padding_value=PAD_IDX)
    return src_batch, tgt_batch

text_transform = {}
for language in [SRC_LANGUAGE, TGT_LANGUAGE]:
    text_transform[language] = sequential_transforms(
        token_transform[language], vocab_transform[language], input_transform
    )

data_iter = Multi30k(split="valid", language_pair=(SRC_LANGUAGE, TGT_LANGUAGE))
dataloader = DataLoader(data_iter, batch_size=BATCH_SIZE, collate_fn=collator)
source_tensor, target_tensor = next(iter(dataloader))

print("(source, target):")
print(next(iter(data_iter)))

print("source_batch:", source_tensor.shape)
print(source_tensor)

print("target_batch:", target_tensor.shape)
print(target_tensor)
```

```
(source, target):
('Eine Gruppe von Männern lädt Baumwolle auf einen Lastwagen', 'A group of men are loading cotton onto a truck')
source_batch: torch.Size([35, 128])
tensor([[ 2,  2,  2, ...,  2,  2,  2],
        [14,  5,  5, ...,  5, 21,  5],
        [38, 12, 35, ..., 12, 1750, 69],
        ...,
        [ 1,  1,  1, ...,  1,  1,  1],
        [ 1,  1,  1, ...,  1,  1,  1],
        [ 1,  1,  1, ...,  1,  1,  1]])
target_batch: torch.Size([30, 128])
tensor([[ 2,  2,  2, ...,  2,  2,  2],
        [ 6,  6,  6, ..., 250, 19,  6],
        [39, 12, 35, ..., 12, 3254, 61],
        ...,
        [ 1,  1,  1, ...,  1,  1,  1],
        [ 1,  1,  1, ...,  1,  1,  1],
        [ 1,  1,  1, ...,  1,  1,  1]])
```

- `sequential_transforms` 함수
 - 여러 개의 전처리 함수를 인자로 받아 이를 차례로 적용하는 함수를 반환하는 함수
 - 예제에서는 세 종류의 전처리 수행
 - 첫 번째 매개변수: `token_transform` 을 적용해 문장을 토큰화
 - 두 번째: `vocab_transform` 을 적용해 각 토큰을 인덱스화
 - 마지막: `input_transform` 함수 - 인덱스화된 토큰에 문장의 시작 `BOS_IDX(2)` 와 끝 `EOS_IDX(3)` 을 알리는 특수 토큰을 할당
- `data_iter` 변수에 (독일어, 영어) 형태로 구성된 `Multi30k` 텍스트 데이터셋 불러옴
 - 이 데이터셋을 데이터로더에 적용, 집합 함수로 `collator` 함수 적용
 - `collator` 함수
 - 배치 단위로 데이터를 처리
 - `rstrip("\n")` 함수로 문자열의 끝에 있는 개행 문자 (`\n`)를 제거
 - `text_transform` 변수에 저장된 `sequential_transforms` 함수를 적용
 - 이후 패딩 시퀀스(`pad_sequence`) 함수를 사용해 소스와 타겟 시퀀스를 패딩
 - 패딩 시퀀스 함수는 동일한 길이를 가지도록 시퀀스의 뒤쪽에 `PAD_IDX(1)` 로 채워진 패딩 토큰을 추가
- 이 데이터로더는 (패딩이 적용된 소스, 패딩이 적용된 타겟) 튜플을 반환
 - 출력되는 배치 데이터 차원은 [소스(타겟) 시퀀스 길이, 배치 크기]를 의미



#어텐션 마스크 생성

```
def generate_square_subsequent_mask(s):
    mask = (torch.triu(torch.ones((s,s), device=DEVICE)) == 1).transpose(0,1)
    mask = (
        mask.float()
        .masked_fill(mask == 0, float("-inf"))
        .masked_fill(mask == 1, float(0.0))
    )
    return mask

def create_mask(src, tgt):
    src_seq_len = src.shape[0]
    tgt_seq_len = tgt.shape[0]

    tgt_mask = generate_square_subsequent_mask(tgt_seq_len)
    src_mask = torch.zeros((src_seq_len, src_seq_len), device=DEVICE).type(torch.bool)

    src_padding_mask = (src == PAD_IDX).transpose(0, 1)
    tgt_padding_mask=(tgt == PAD_IDX).transpose(0, 1)
    return src_mask, tgt_mask, src_padding_mask, tgt_padding_mask

target_input = target_tensor[:-1, :]
target_out = target_tensor[1:, :]

source_mask, target_mask, source_padding_mask, target_padding_mask = create_mask(
    source_tensor, target_input
)

print("source_mask:", source_mask.shape)
print(source_mask)
print("target_mask:", target_mask.shape)
print(target_mask)
print("source_padding_mask:", source_padding_mask.shape)
print(source_padding_mask)
print("target_padding_mask:", target_padding_mask.shape)
print(target_padding_mask)
```

출력 결과

```
source_mask: torch.Size([35, 35])
tensor([[False, False, False, ..., False, False, False],
        [False, False, False, ..., False, False, False],
        ...,
        [False, False, False, ..., False, False, False],
        [False, False, False, ..., False, False, False]], device='cuda:0')
target_mask: torch.Size([29, 29])
tensor([[0., -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf,
        -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf,
        -inf, -inf, -inf, -inf],
        [0., 0., -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf,
        -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf, -inf,
        -inf, -inf, -inf, -inf],
        ...,
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
        0.,
        0., 0., 0., 0., -inf],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
        0.,
        0., 0., 0., 0., 0.]], device='cuda:0')
source_padding_mask: torch.Size([128, 35])
tensor([[False, False, False, ..., True, True, True],
        [False, False, False, ..., True, True, True],
        ...,
        [False, False, False, ..., True, True, True],
        [False, False, False, ..., True, True, True]])
```

```
...,
[False, False, False, ..., True, True, True],
[False, False, False, ..., True, True, True]])
target_padding_mask: torch.Size([128, 29])
tensor([[False, False, False, ..., True, True, True],
        [False, False, False, ..., True, True, True],
        ...,
        [False, False, False, ..., True, True, True],
        [False, False, False, ..., True, True, True]])
```

- 트랜스포머 모델에서 사용되는 마스크를 생성하는 함수, 두 시퀀스의 패딩 마스크를 생성하는 함수로 구성됨
- `generate_square_subsequent_mask` : 마스크를 생성하는 함수
 - 입력으로 정수 `s` 를 받아 `sxs` 크기의 마스크를 생성
 - `torch.ones` 함수를 사용해 1로 채워진 행렬을 만든 후 `torch.triu` 함수를 적용해 상삼각행렬 생성

- `transpose` 함수 사용하여 행렬을 전치
- 마스크 텐서에서 0인 값은 `-inf`로, 1인 값은 `0.0`으로 채워 어텐션 연산에 적용
 - `0.0`은 셀프 어텐션에 참조되는 시퀀스 가리킴
 - `-inf` 값은 셀프 어텐션 계산 과정에서 어텐션 스코어가 0에 수렴하기 때문에 해당 타깃 입력 시퀀스를 제외시키는 역할 함
- `create_mask` : 패딩 마스크를 생성하는 함수
 - 시퀀스를 입력받아 길이를 계산하고 마스크 생성 함수로 타깃 시퀀스의 마스크를 생성
 - 소스 마스크 - 소스 시퀀스 길이의 크기로 채워진 행렬 생성, 패딩 마스크 - 각 시퀀스에 대해 패딩 토큰의 위치를 찾고 `transpose` 함수로 전치
 - 패딩 마스크 생성 전에 타깃 데이터의 입력값(`target_input`)과 출력값(`target_out`)은 토큰 순서를 한 칸 시프트하여 이전 토큰들이 주어졌을 때 다음 토큰을 예측하게 함
- 패딩 마스크 생성 함수로 소스 시퀀스 입력값(`source_tensor`)과 타깃 시퀀스 입력값(`target_input`)을 전달해 4개의 텐서 생성
 - `source_mask`
 - 셀프 어텐션 과정에서 참조되는 소스 데이터의 시퀀스 범위
 - `False` 인 위치는 셀프 어텐션에 참조되는 토큰, `True` 인 위치는 제외되는 토큰
 - 출력 결과 확인해 보면 소스 데이터는 모든 시퀀스를 대상으로 셀프 어텐션이 수행됨
 - `target_mask`
 - [쿼리 시퀀스 길이, 키 시퀀스 길이]의 형태로 구성
 - i 번째 쿼리 벡터는 $i+1$ 이상의 키 벡터에 대해 어텐션 연산을 수행할 수 없게 됨
 - ⇒ 모델이 현재 예측하고자 하는 위치 이전의 토큰들만 참고하게 제한 → 모델이 미래 시점의 정보를 사용하지 않게 해 현재 시점에 영향 미치지 않게 함
 - `source_padding_mask`, `target_padding_mask`
 - 소스(타깃) 배치 데이터에서 텍스트 토큰이 존재하는지 여부 나타냄

- `False` 인 경우 해당 토큰 인덱스가 존재, `True` 인 경우 해당 토큰 인덱스가 패딩 토큰으로 채워져 있음



#모델 학습 및 평가

```
def run(model, optimizer, criterion, split):
    model.train() if split == "train" else model.eval()
    data_iter = Multi30k(split=split, language_pair=(SRC_LANGUAGE, TGT_LANGUAGE))
    dataloader = DataLoader(data_iter, batch_size=BATCH_SIZE, collate_fn=collator)

    losses = 0
    for source_batch, target_batch in dataloader:
        source_batch = source_batch.to(DEVICE)
        target_batch = target_batch.to(DEVICE)

        target_input = target_batch[:-1, :]
        target_output = target_batch[1:, :]

        src_mask, tgt_mask, src_padding_mask, tgt_padding_mask = create_mask(
            source_batch, target_input
        )

        logits = model(
            src=source_batch,
            trg=target_input,
            src_mask=src_mask,
            tgt_mask=tgt_mask,
            src_padding_mask=src_padding_mask,
            tgt_padding_mask=tgt_padding_mask,
            memory_key_padding_mask=src_padding_mask,
        )

        optimizer.zero_grad()
        loss = criterion(logits.reshape(-1, logits.shape[-1]), target_output.reshape(-1))
        if split == "train":
            loss.backward()
            optimizer.step()
        losses += loss.item()

    return losses / len(list(dataloader))

for epoch in range(5):
    train_loss = run(model, optimizer, criterion, "train")
    val_loss = run(model, optimizer, criterion, "valid")
    print(f"Epoch: {epoch+1}, Train loss: {train_loss:.3f}, Val loss: {val_loss:.3f}")
```

출력 결과

```
Epoch: 1, Train loss: 4.462, Val loss: 4.040
Epoch: 2, Train loss: 3.553, Val loss: 3.721
Epoch: 3, Train loss: 3.305, Val loss: 3.686
Epoch: 4, Train loss: 3.154, Val loss: 3.488
Epoch: 5, Train loss: 3.068, Val loss: 3.401
```

- `run` 함수

- 모델 학습 및 평가를 위한 함수
- 소스와 타겟 데이터를 입력받아 `collator` 로 문장들을 토큰화, 인덱스로 변환
- `create_mask` 함수
 - 트랜스포머 모델에 필요한 입력 패딩 마스크(`src_padding_mask` , `tgt_padding_mask`)와 어텐션 마스크(`src_mask` , `tgt_mask`)를 생성
 - 결괏값들은 타겟 시퀀스의 i번째까지 토큰이 주어졌을 때 i+1번째 토큰을 예측하는 데 활용됨
- 출력 결과 보면 Train loss와 Val loss 모두 감소해 모델이 학습됨



#트랜스포머 모델 번역 결과

```
def greedy_decode(model, source_tensor, source_mask, max_len, start_symbol):
    source_tensor = source_tensor.to(DEVICE)
    source_mask = source_mask.to(DEVICE)

    memory = model.encode(source_tensor, source_mask)
    ys=torch.ones(1,1).fill_(start_symbol).type(torch.long).to(DEVICE)
    for i in range(max_len - 1):
        memory = memory.to(DEVICE)
        target_mask = generate_square_subsequent_mask(ys.size(0))
        target_mask= target_mask.type(torch.bool).to(DEVICE)

        out = model.decode(ys, memory, target_mask)
        out = out.transpose(0, 1)
        prob = model.generator(out[:, -1])
        _, next_word = torch.max(prob, dim=1)
        next_word = next_word.item()

        ys = torch.cat(
            [ys, torch.ones(1,1).type_as(source_tensor.data).fill_(next_word)], dim=0
        )
        if next_word == EOS_IDX:
            break
    return ys

def translate(model, source_sentence):
    model.eval()
    source_tensor = text_transform[SRC_LANGUAGE](source_sentence).view(-1, 1)
    num_tokens = source_tensor.shape[0]
    src_mask = (torch.zeros(num_tokens, num_tokens)).type(torch.bool)
    tgt_tokens = greedy_decode(
        model, source_tensor, src_mask, max_len=num_tokens + 5, start_symbol=BOS_IDX
    ).flatten()
    output = vocab_transform[TGT_LANGUAGE].lookup_tokens(list(tgt_tokens.cpu().numpy()))[1:-1]
    return " ".join(output)

output_oov = translate(model, "Eine Gruppe von Menschen steht vor einem Iglu .")
output = translate(model, "Eine Gruppe von Menschen steht vor einem Gebäude .")
print(output_oov)
print(output)
```

출력 결과

```
A group of people are standing in front of a large building .
A group of people are standing in front of a large building .
```

- 모델 번역 방식은 **그리드 디코딩(Greedy decoding)** 방식으로 번역 결과를 출력
 - 디코더 네트워크가 생성한 확률 분포에서 가장 높은 확률을 가지는 단어 선택하는 방법

- 현재 시점에서 가장 확률이 높은 단어를 선택해 디코딩 진행
- 모델 추론 방식은 디코더에 참조되는 마지막 인코더 트랜스포머 블록의 벡터 (`memory`), 타겟 데이터의 입력 텐서(`ys`), 타겟 마스크(`target_mask`)를 사용
 - `memory` 텐서를 생성하려면 소스 문장을 토큰 인덱스로 표현한 `source_tensor` 를 생성하고, `source_mask` 는 소스 문장에서 모든 토큰이 어텐션 될 수 있게 0 값으로 설정
 - `model.encode` 메서드에 `source_tensor` 와 `source_mask` 를 입력으로 넣어 소스 문장에 대한 인코딩을 수행해 마지막 인코더 트랜스포머 블록의 벡터 추출
 - `model.decode` 메서드에서 생성된 out은 [토큰 개수, 배치 크기, 확률]의 형태
 - 이 값을 `transpose` 함수 이용하여 [배치 크기, 토큰 개수, 확률] 형태로 변환 후 텐서를 슬라이싱해 [배치 크기, 확률] 형태로 만듦
 - 이후 어휘 사전에서 가장 확률이 높은 토큰 인덱스 찾기
- ⇒ 이 과정을 `max_len` 이전이거나 `EOS_IDX` 예측할 때까지 반복, 최종적으로 예측된 토큰 시퀀스를 반환
- `max_len` 은 `num_tokens + 5` 로 구성
 - `num_tokens` 는 소스 문장의 토큰 개수 의미
 - 생성된 문장의 길이가 소스 문장 길이보다 약간 더 길어지는 경우 많기 때문에 5를 더함
- `greedy_decode` 함수
 - 현재까지 예측된 토큰들 이용해 가장 높은 확률 가지는 단어 선택하여 다음 토큰으로 예측
 - 예측된 토큰 인덱스를 어휘 사전의 `lookup_tokens` 함수를 통해 텍스트로 변환
 - 후처리로 `<bos>` 와 `<eos>` 토큰은 슬라이싱 (`[1:-1]`)으로 제거
 - 처리된 텍스트를 공백으로 구분해 하나의 문장으로 만든 후 반환
- 번역 결과를 보면 모델이 입력 문장의 의미 정확히 파악 못하고 잘못된 번역 결과 도출
 - 이글루Iglu는 OOV 데이터로 학습하지 않았기 때문에 번역 결과가 부정확할 수 있음

- 건물Gebäude을 입력해 번역하면 모델이 입력 문장의 의미 올바르게 파악
 - 출력 결과 보면 OOV인 Iglu도 'building'으로 번역했기 때문에 Gebäude를 인식해 'building'으로 번역한 것이 아닌 'large' 다음에 자연스럽게 'building'이 따라올 것이라는 걸 학습했을 가능성
- ⇒ 더 정확한 번역 위해 더 많은 학습 데이터 사용하거나 모델의 구조 또는 하이퍼파라미터 등을 변경해 모델의 성능을 지속적으로 모니터링, 개선

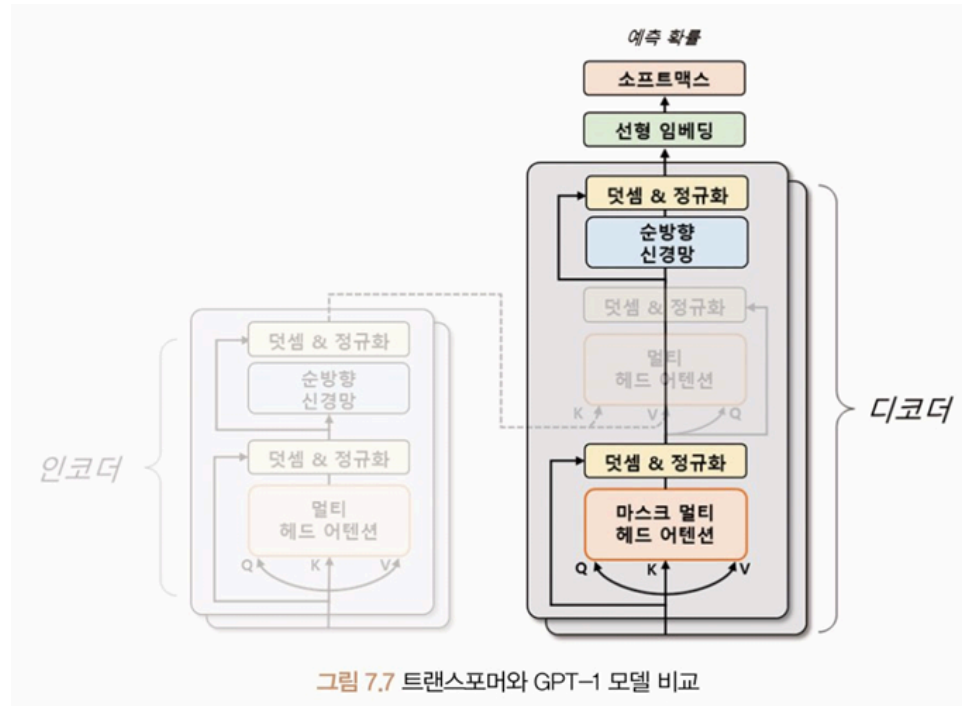
GPT

- **GPT(Generative Pre-trained Transformer)**
 - 2018년 OpenAI에서 발표한 트랜스포머 기반 언어 모델
 - ELMo(Embeddings from Language Model)와 같이 대규모 말뭉치로 사전 학습된 모델로, 다양한 다운스트림 작업에서 우수한 성능
- GPT 모델은 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델
 - 인코더는 입력 문장의 특징 추출에 초점 두고 있다면, 디코더는 입력 문장과 출력 문장 간의 관계를 이해하고 출력 문장을 생성하는 데 초점
 - 디코더를 쌓아 모델 구성하는 것이 자연어 생성과 같은 언어 모델링 작업에서 더 높은 성능
 - 대규모 텍스트 데이터세트로 사전 학습해 초기화되며, 특정 자연어 처리 작업에 맞게 미세 조정해 사용
- 자연어 생성, 기계 번역, 질의응답, 요약 등 다양한 자연어 처리 작업에 활용
- 예측 성능 뛰어나고 적은 데이터로도 높은 성능
- 다양한 시리즈 존재
 - GPT-1(2018), GPT-2(2019), GPT-3(2020)
 - 거의 동일한 트랜스포머 구조, 버전 갱신될수록 더 많은 트랜스포머 계층과 데이터 활용해 학습
 - GPT-1: 약 1억 2천만 개의 매개변수 → GPT-2: GPT-1의 성능 크게 능가, 약 15억 개의 매개변수 → GPT-3: 약 1,750억 개의 매개변수 갖는 초거대 모델, 매우 뛰어난 문장 생성 성능, 많은 자원 필요

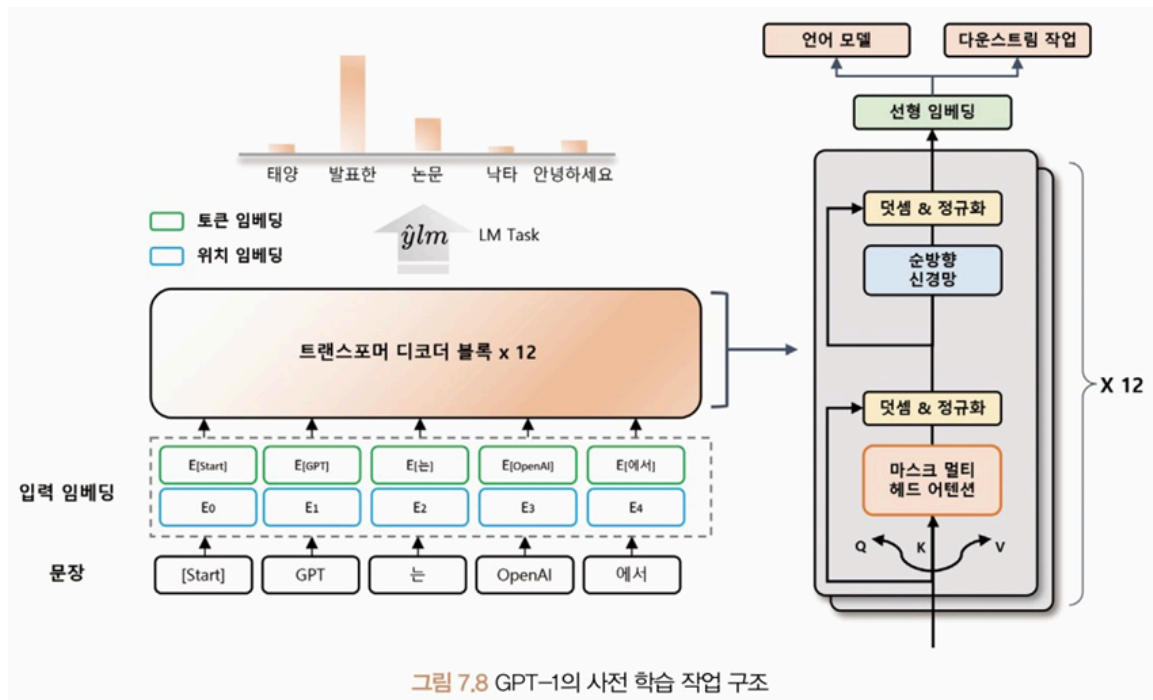
- 많은 양의 매개변수를 줄이기 위해 OpenAI는 **RLHF(Reinforcement Learning from Human Feedback)** 방법 도입 → GPT-3.5 모델 학습
 - RLHF
 - 인간의 피드백을 이용하는 강화 학습 방법
 - 인간이 모델이 생성한 결과물을 평가, 모델이 좋은 결과물 생성하면 보상 주어 모델을 학습
 - 기존의 학습 데이터만 사용하는 것보다 더욱 정확, 효율적 학습 가능
→ 약 60억 개의 가중치로도 뛰어난 성능
 - 2022.10 ChatGPT 서비스 공개
 - GPT-3.5 기반, 자연스러운 대화 생성 및 사용자와의 대화 통해 지속적으로 학습
 - 2023.3 GPT-4 모델 발표
 - 텍스트 데이터에 국한되지 않고 이미지 데이터도 처리 가능
-

GPT-1

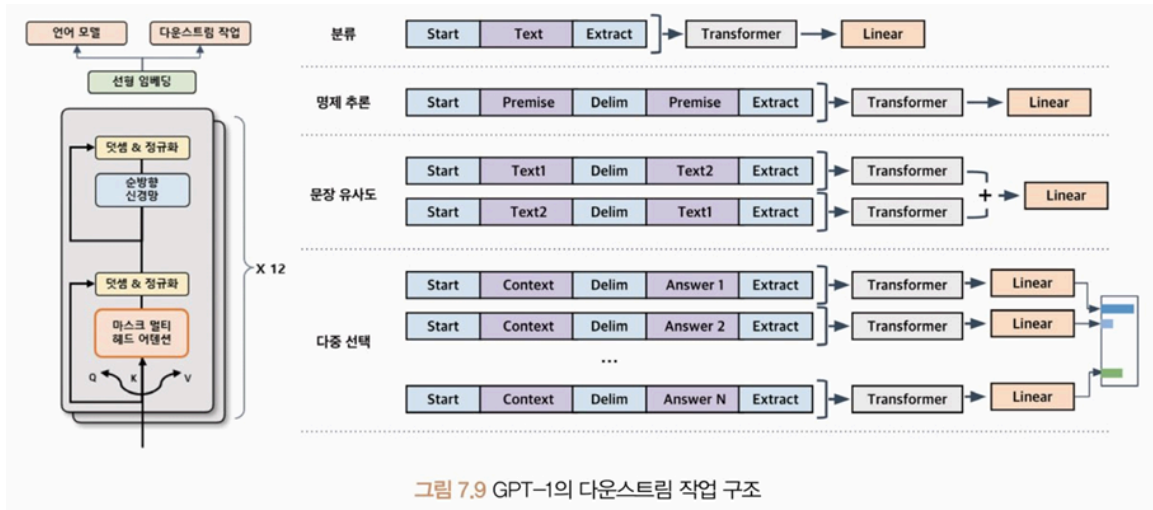
- 트랜스포머 구조를 바탕으로 한 **단방향(Unidirectional)** 언어 모델
 - 텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측하는 모델
 - 이전 단어들을 차례대로 입력하면서 다음 단어를 예측하는 방식으로 동작
 - 모델의 계산량 줄어들어 학습 속도 빠르고, 모델의 크기 작아질 수 있단 장점
- GPT-1은 이전 단어들에 대한 정보로 다음 단어를 예측할 때 트랜스포머 구조 활용
 - 디코더 부분만 사용, 인코더-디코더 간 어텐션 연산 제외
→ 매개변수 수를 줄이면서도 높은 성능



- 디코더의 두 번째 다중헤드 어텐션은 사용 X
- 12개의 헤드를 가진 트랜스포머의 디코더를 12층 쌓아 모델 구성, 약 1억 2천만개의 매개변수로 학습됨
- 약 4.5GB의 BookCorpus 데이터세트를 사용해 언어 모델을 사전 학습
 - 입력 문장을 임의의 위치까지 보여주고 뒤에 이어지는 문장을 모델이 자동으로 예측하게 함
 - 별도의 레이블이 필요하지 않기 때문에 비지도 학습 이루어짐
 - 컴퓨팅 자원만 충분하다면 제약 없이 학습 가능하다는 장점



- 입력 문장의 일부만 보고 다음 토큰을 예측하도록 하는 언어 모델을 통해 사전 학습
 - 입력 문장을 일정한 길이의 블록으로 나누어 처리
 - 각 블록에서 블록 내의 첫 번째 토큰부터 순서대로 다음 토큰 예측하게 함
→ 모델이 장기적인 문맥 이해 가능
- GPT-1의 입력 임베딩은 토큰 임베딩과 위치 임베딩을 이용해 계산됨
 - 토큰 임베딩: 각 토큰을 고정된 차원의 벡터로 매핑
 - 위치 임베딩: 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터를 매핑
- 미세 조정을 위한 다운스트림 작업에서도 각 작업에 따라 입출력 구조 바꾸지 않고 언어 모델을 **보조 학습(Auxiliary Learning)**해 사용 가능
 - 보조 학습: 모델이 주어진 주요 학습 작업 외에도 다른 작은 부가적인 학습 작업을 동시에 수행하는 것
 - 보조 작업에서 학습한 정보를 주요 작업에 전달하여 성능 향상 가능



- 원하는 목표에 맞게 모델을 세밀하게 조정해야 함 → 다운스트림 작업에서 추가 손실 함수 필요
- 두 개의 손실 함수를 사용해 최적화하면 보조 학습을 통해 언어 모델 개선하는 동시에 다운스트림 작업의 목표를 달성하기 위해 필요한 정보 학습 가능
- GPT-1 다운스트림 작업에서도 특수 토큰 사용
 - 문장의 시작 의미하는 `<start>` 토큰, 두 문장의 경계 의미하는 `<delim>` 토큰, 문장의 마지막 의미하는 `<extract>` 토큰

GPT-2

- GPT-1: 입력 문장 최대 512의 길이, 12개의 디코더 층, 1,200만 개의 매개변수
 ↔ GPT-2: 1,024의 길이, 48개의 디코더 계층, 15억 개의 매개변수

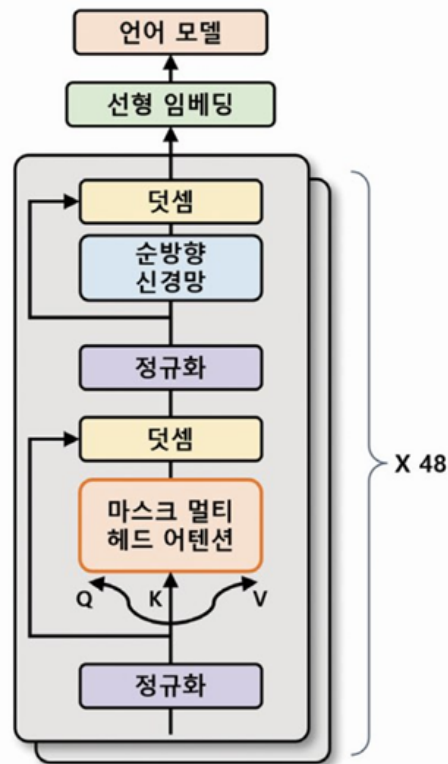


그림 7.10 GPT-2의 모델 구조

- GPT-1보다 훨씬 더 많은 양의 데이터 이용해 사전 학습
 - 미국의 웹사이트에서 스크래핑한 약 8백만 개의 문서 이용해 약 40GB 데이터로 사전 학습 수행
- 제로-샷 학습 가능
 - 별도의 미세 조정 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습 과정에서 특정한 형식의 데이터 입력
→ 학습 과정으로 배우지 않은 작업에도 사용 가능

GPT-3

- GPT-2와 거의 동일한 모델 구조, 몇몇 매개변수의 변화로 약 116배 많은 매개변수
- 매우 규모 큰 모델로 96개의 헤드 가진 멀티 헤드 어텐션을 사용, 96개의 트랜스포머 디코더 층 사용
 - 멀티 헤드 어텐션 과정에서 연산량을 줄이기 위해 **희소 어텐션(Sparse Attention)**과 일반 어텐션을 섞어 사용
- 토큰 임베딩의 크기도 1,600 → 12,888로 증가

- GPT-3는 2,048개의 토큰까지 입력 가능, 더욱 긴 문장에 대한 처리 능력 향상
- 웹 크롤링, 위키백과, 서적 등에서 수집한 약 45TB의 데이터세트 이용해 모델 학습
 - 다운스트림 작업으로 학습하지 않았지만, 질의응답, 번역, 요약, 문서 생성 등 다양한 다운스트림 작업에서도 높은 성능 보임
- **프롬프트(prompt)** 형식으로 작업을 수행

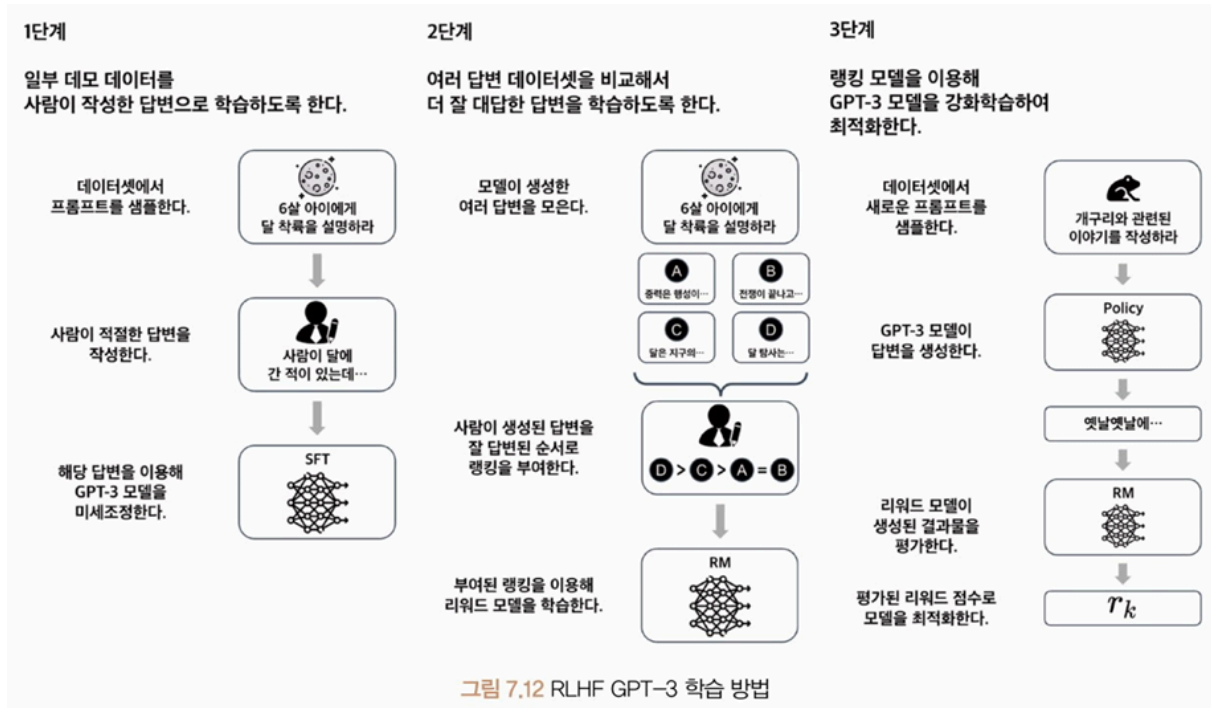
질의 응답 입력 프롬프트 문서 : 생성적 사전학습 트랜스포머(Generative Pre-Trained Transformer 3), GPT-3는 OpenAI에서 만든 대형 언어 모델이다. 질문 : GPT-3를 만든 곳은 어디인가? 답변 : OpenAI 출력값	자연어 추론 입력 프롬프트 자연어 추론 : OpenAI는 미국의 인공지능 연구소로, 인류에게 이익을 주는 것을 목표로 한다. GPT-3등 거대 언어 모델을 개발하여 많은 이목을 끌었다. 질문 : OpenAI는 상장사인가? 참, 거짓, 둘 다 아님 답변 : 둘 다 아님 출력값
수학 문제 풀이 입력 프롬프트 질문 : $(2 * 4) * 6$은 무엇인가? 답변 : 48 출력값	일반 상식 질의 응답 입력 프롬프트 질문 : 물은 섭씨 몇 도에서 어는가? 답변 : 0도 출력값

그림 7.11 GPT-3의 다운스트림 작업 프롬프트 예시

- 새로운 도메인의 작업에 대해서 높은 일반화 능력, 기존 자연어 처리 분야에서의 기술적 한계 극복
- 큰 모델이기 때문에 매우 느리고 비용이 많이 들어 일반 사용자나 소규모 기업에서 활용 어려움
- 사전 학습된 데이터에 기반하여 작동 → 데이터가 편향되어 있거나 정확하지 않은 경우 오류 발생

GPT 3.5

- GPT-3의 모델 구조 따르면서도 새로운 학습 방법인 RLHF 도입해 모델의 매개변수 수 줄이고 자연스러움 늘림
- 입력으로 최대 4,096개의 토큰 처리 가능



- 데이터셋에서 임의의 프롬프트 가져오고 이 프롬프트에 대해 사람이 직접 적절한 답을 작성하게 하여 학습
 - 지도 미세 조정(Supervised Fine-Tuning, SFT)를 통해 학습
 - 작성한 답변을 모델이 생성한 문장과 함께 평가해 모델이 생성한 문장이 자연스러우면 보상받고, 그렇지 않으면 보상받지 못하는 강화 학습 통해 모델 학습
 - GPT 모델이 얼마나 잘 답변했는지 평가하는 보상 모델을 학습
 - 시드(seed)에 따라 다른 답변 생성하므로 내용이 다른 여러 개의 답변 생성 가능
 - 보상 모델로 사람이 생성된 답변의 우수성을 평가하고 그에 따른 랭킹 부여, 보상 모델의 입력으로 사용
 - 생성된 답변과 랭킹 정보를 활용하여 학습해 더 자연스러운 문장 생성 가능
- RLHF에서는 주어진 프롬프트에 대해 GPT-3 언어 모델이 답변을 생성하고, 이를 이용해 사람의 피드백을 기반으로 한 학습된 보상 모델을 평가
 - 생성된 답변과 랭킹 정보를 활용하여 리워드 모델이 학습, 이를 통해 더욱 자연스러운 문장 생성이 가능해지는 GPT-3.5가 학습됨 → 과정 반복하여 더 정교한 답변 생성 모델 학습

GPT-4

- 텍스트 데이터뿐만 아니라 이미지 데이터도 인식 간으한 **멀티모달(Multimodal)** 모델

- GPT-3.5의 최대 여덟 배인 32,768개의 토큰 입력 가능
 - 대규모 다중 작업 언어 이해 벤치마크 테스트 결과 85%까지 성능 향상 등 GPT-3.5 보다 높은 성능 향상
 - 언어 모델이 일부 입력에 대해 현실적이지 않은 결과를 생성하는 **환각(Hallucination)** 현상이 아직 존재
 - 모델이 인간과 같은 추론과 판단 능력을 갖추지 못하고 부적절하거나 오류가 많은 답변을 제시하는 현상
 - 모델의 매개변수 수나 학습 데이터 및 방법은 공개 X
-

모델 실습

- 허깅 페이스 트랜스포머스 라이브러리의 GPT-2 모델로 문장 생성



#문장 생성을 위한 GPT-2 모델의 구조

```
from transformers import GPT2LMHeadModel

model = GPT2LMHeadModel.from_pretrained(pretrained_model_name_or_path="gpt2")

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("| | L", sssub_name)

transformer
L wte
L wpe
L drop
L h
| L 0
| | L ln_1
| | L attn
| | L ln_2
| | L mlp
| L 1
| | L ln_1
| | L attn
| | L ln_2
| | L mlp
| L 2
| | L ln_1
| | L attn
| | L ln_2
| | L mlp
| L 11
| | L ln_1
| | L attn
| | L ln_2
| | L mlp
L ln_f
lm_head
```

- 트랜스포머 라이브러리의 `GPT2LMHeadModel` 클래스의 `from_pretrained` 메서드로 사전 학습된 GPT-2 모델 불러오기
 - `pretrained_model_name_or_path` 매개변수: 불러오려는 사전 학습된 모델
- 실습 모델로 12개의 디코더 계층을 사용하는 간소화 모델 `gpt2` 를 사용
 - 48개의 디코더 계층 사용하는 모델은 `gpt2-xl`

- 단어 토큰 임베딩(`wte`), 단어 위치 임베딩(`wpe`), 드롭아웃(`drop`), 트랜스포머 디코더 계층(`h`), 선형 임베딩 및 언어 모델(`lm_head`)로 구성
- 트랜스포머 라이브러리는 문장 분류, 문장 생성, 토큰 분류 등 다양한 작업에 대한 전처리, 모델 아키텍처, 후처리를 각각 처리할 수 있는 파이프라인 함수를 제공



```
#GPT-2를 이용한 문장 생성
from transformers import pipeline

generator = pipeline(task="text-generation", model="gpt2")
outputs = generator(
    text_inputs="Machine learning is",
    max_length=20,
    num_return_sequences=3,
    pad_token_id=generator.tokenizer.eos_token_id
)
print(outputs)
```

출력 결과

```
[{'generated_text': 'Machine learning is the ability to model and learn from the world around you. That knowledge is fundamental to'},
 {'generated_text': 'Machine learning is one of the earliest artificial intelligence projects. It has already been used in machine learning to'},
 {'generated_text': 'Machine learning is a fundamental component of our approach to artificial intelligence, based on deep learning. We'll'}]
```

- 파이프라인 클래스는 입력된 작업(`task`)에 모델(`model`)로 적합한 파이프라인을 구축
 - 작업 매개변수는 수행하려는 작업을 의미
- 파이프라인 함수는 설정한 작업을 수행하는 파이프라인 클래스의 인스턴스를 반환
 - 텍스트 생성 작업을 의미하는 `"text-generation"` 작업을 입력하면 `TextGenerationPipeline` 클래스의 인스턴스가 생성됨
 - 입력 텍스트(`text_inputs`): 생성하려는 문장의 입력 문맥 전달
 - 최대 길이(`max_length`): 생성될 문장의 최대 토큰 수 제한
 - 반환 시퀀스 개수(`num_return_sequences`): 생성할 텍스트 시퀀스의 수 의미
 - 패딩 토큰 ID(`pad_token_id`): 모델의 **자유 생성(Open-end generation)** 여부 설정
 - 사용하면 모델이 입력된 문장의 문맥을 기반으로 자유롭게 다음 단어나 문장을 생성

- GPT-2 모델은 선형 임베딩 층을 이용해 텍스트 분류 등 다양한 다운스트림 작업에 활용 가능
- CoLA(The Corpus of Linguistic Acceptability) 데이터셋을 이용해 모델 학습



```
#CoLA 데이터셋 불러오기
import torch
from torchtext.datasets import CoLA
from transformers import AutoTokenizer
from torch.utils.data import DataLoader

def collator(batch, tokenizer, device):
    source, labels, texts = zip(*batch)
    tokenized = tokenizer(
        texts,
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(labels, dtype=torch.long).to(device)
    return input_ids, attention_mask, labels

train_data = list(CoLA(split="train"))
valid_data = list(CoLA(split="dev"))
test_data = list(CoLA(split="test"))

tokenizer = AutoTokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token

epochs = 3
batch_size = 16
device = "cuda" if torch.cuda.is_available() else "cpu"

train_dataloader = DataLoader(
    train_data,
    batch_size=batch_size,
    collate_fn=lambda x: collator(x, tokenizer, device),
    shuffle=True,
)

valid_dataloader = DataLoader(
    valid_data, batch_size=batch_size, collate_fn=lambda x: collator(x, tokenizer, device)
)

test_dataloader = DataLoader(
    test_data, batch_size=batch_size, collate_fn=lambda x: collator(x, tokenizer, device)
)

print("Train Dataset Length :", len(train_data))
print("Valid Dataset Length :", len(valid_data))
print("Test Dataset Length :", len(test_data))
```

```
Train Dataset Length : 8550
Valid Dataset Length : 526
Test Dataset Length : 515
```

- CoLA 데이터세트는 `train, dev, test` 로 구성
 - 차례대로 모델 학습, 모델 검증, 학습된 모델의 성능 평가에 사용
- GPT-2는 사전 학습 시 패딩 기법을 사용하지 않기 때문에 토큰라이저의 특수 토큰 중 패딩 토큰이 따로 포함되어 있지 않음
 - 문장 분류 모델을 학습하기 위해 문장의 끝을 의미하는 `eos` 토큰으로 패딩 토큰을 대체
- `collator` 함수
 - 배치를 토큰라이저로 토큰화
 - 패딩(`padding`), 절사(`truncation`), 반환 형식 설정(`return_tensors`) 작업을 수행
 - 패딩에 인자를 `longest` 로 설정하면 가장 긴 시퀀스에 대해 패딩 적용
 - 절사에 인자를 `True` 로 설정하면 입력 시퀀스 길이가 최대 길이 초과하는 경우 해당 시퀀스를 자름
 - 반환 형식 설정에 `pt` 를 입력하면 파이토치 텐서 형태로 결과 반환
- 토큰라이저는 토큰 ID(`input_ids`)와 어텐션 마스크(`attention_mask`)를 반환
 - 어텐션 마스크를 이용해 입력 문장에서 패딩 부분을 무시하고 실제 단어에 대해 처리
 - 토큰 ID: 토큰라이저가 각 토큰에 대해 부여한 숫자 ID 담고 있음
 - 어텐션 마스크: 입력 문장의 실제 단어에 대응하는 부분은 1로, 패딩에 대응하는 부분은 0으로 채움



```
#GPT-2 모델 설정
from torch import optim
from transformers import GPT2ForSequenceClassification

model = GPT2ForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="gpt2",
    num_labels=2
).to(device)
model.config.pad_token_id = model.config.eos_token_id
optimizer = optim.Adam(model.parameters(), lr=5e-5)
```

- GPT-2 문장 분류 모델(`GPT2ForSequenceClassification`) 클래스
 - GPT-2 모델을 기반으로 하는 시퀀스 분류 모델
 - 최종 출력 계층이 분류를 위해 미세 조정됨
- `CoLA` 데이터세트는 올바른 문장과 올바르지 않은 문장이 태깅되어 있으므로 분류 레이블 수(`num_labels`)를 2로 설정
- GPT-2는 사전 학습 시 토큰의 패딩 기법 사용 X → 토크나이저에 패딩 토큰 포함 X
 - 하지만 문장 분류 모델을 학습하기 위해서는 모델이 고정된 길이의 입력을 필요로 하므로 문장의 끝을 나타내는 `eos` 토큰을 사용해 패딩 토큰 대체
 - 문장 분류 모델에서 필요로 하는 고정된 길이의 입력 제공 가능



#GPT-2 모델 학습 및 검증

```
import numpy as np
from torch import nn

def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0
    for input_ids, attention_mask, labels in dataloader:
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        loss = outputs.loss
        train_loss += loss.item()

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    train_loss = train_loss / len(dataloader)
    return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        criterion = nn.CrossEntropyLoss()
        val_loss, val_accuracy = 0.0, 0.0

        for input_ids, attention_mask, labels in dataloader:
            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )
            logits = outputs.logits

            loss = criterion(logits, labels)
            logits=logits.detach().cpu().numpy()
            label_ids = labels.to("cpu").numpy()
            accuracy = calc_accuracy(logits, label_ids)

            val_loss += loss
            val_accuracy += accuracy

        val_loss = val_loss/len(dataloader)
        val_accuracy = val_accuracy/len(dataloader)
        return val_loss, val_accuracy

best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val Accuracy: {val_accuracy:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "../models/GPT2ForSequenceClassification.pt")
        print("Saved the model weights")
```

```
Epoch 1: Train Loss: 0.5071 Val Loss: 0.5895 Val Accuracy: 0.7316
Saved the model weights
Epoch 2: Train Loss: 0.3701 Val Loss: 0.5130 Val Accuracy: 0.7771
Saved the model weights
Epoch 3: Train Loss: 0.2531 Val Loss: 0.5594 Val Accuracy: 0.8019
```

- GPT-2 문장 분류 모델 클래스로 학습하면 내부적으로 손실을 계산해 반환하므로 `train` 함수로 모델 학습 시 손실은 모델 출력값(`outputs`)의 `loss` 속성으로 가져옴
- 교차 엔트로피 함수로 모델 평가
 - 모델에서 반환하는 손실값이 아닌 로짓(`logits`) 값과 레이블로 손실 계산
- `calc_accuracy` 함수로 모델의 정확도 계산
 - 모델의 예측 결과와 실제 레이블을 비교하여 정확도를 계산하고 반환
- 각 에폭에서 학습 손실, 검증 손실, 검증 정확도 계산하고 검증 손실값이 가장 낮은 값을 갖는 체크포인트 저장
- 모델 선택 방법은 초기 `best_loss` 를 큰 값으로 설정한 후, 각 에폭마다 검증 손실값이 그 값보다 작으면 모델이 개선되었다 판단, 해당 시점의 모델 가중치 저장



#모델 평가

```
model = GPT2ForSequenceClassification.from_pretrained(  
    pretrained_model_name_or_path="gpt2",  
    num_labels=2  
)  
.to(device)  
model.config.pad_token_id = model.config.eos_token_id  
model.load_state_dict(torch.load("../models/GPT2ForSequenceClassification.pt"))
```

```
test_loss, test_accuracy = evaluation(model, test_dataloader)  
print(f"Test Loss : {test_loss:.4f}")  
print(f"Test Accuracy : {test_accuracy:.4f}")
```

```
Test Loss : 0.6545  
Test Accuracy : 0.7279
```

- 손실 및 정확도가 중간 정도의 수치 → 대체로 예측 잘 하지만 개선 필요
- 사용한 학습 데이터세트는 8,550개로 모델 규모에 비해 매우 작은 데이터세트 사용했음에도 높은 성능 발휘하는 것 볼 수 있음 → 모델이 작은 데이터세트에서도 일반화 능력 갖고 있음